

VIBKED — Prototype Capabilities & Future Roadmap

Your Private AI OS • Capability document for self-awareness / internal knowledge

1. What VIBKED Is

VIBKED is a local-first private AI knowledge system built around **DATA** → **KNOWLEDGE** → **MEMORY** → **REASONING** → **ACTION**. The current prototype validates document ingestion, semantic retrieval, RAG, and local-model reasoning.

2. Current Prototype Capabilities

Capability	Current status / behavior
Local LLM	Qwen3 8B runs locally through Ollama; the RAG pipeline calls the local Ollama chat API.
Document ingestion	Supports PDF, DOCX, XLSX, TXT, Markdown, CSV, and JSON in the current ingestion pipeline.
PDF extraction	Extracts text page-by-page and preserves source filename and page metadata.
DOCX extraction	Extracts non-empty paragraphs and preserves source metadata.
XLSX extraction	Reads workbook sheets and converts populated rows into searchable text with sheet metadata.
TXT / MD / CSV / JSON	Reads these formats and converts their contents into searchable text.
Chunking	Uses 500-character chunks with 50-character overlap by default.
Embeddings	Uses Ollama with nomic-embed-text; the project uses 768-dimensional embeddings.
Vector database	Persists embeddings, chunks, documents, and metadata locally in ChromaDB.
Semantic retrieval	Embeds a query and retrieves semantically similar chunks from ChromaDB.
Relevance filtering	Filters clearly weaker results using a relative distance heuristic.
Multi-document retrieval	Can combine relevant chunks from multiple documents.
RAG	Builds retrieved context and sends it to Qwen3 for an answer.
Answer modes	Current RAG supports document, general, and hybrid modes.
Source tracking	Returns source labels such as filename + page or filename + sheet.
FastAPI foundation	A FastAPI app and basic root endpoint have been created; API integration is the next stage.

3. Current File Support

Supported: PDF, DOCX, XLSX, TXT, MD, CSV, JSON. **PPTX:** intentionally paused for now; simple slide-text extraction is not part of the current supported ingestion set.

4. Current Architecture

Layer	Component	Role
Input	Document loader	Reads files and attaches source metadata.
Knowledge	Chunker	Splits content into overlapping chunks.
Embeddings	Ollama + nomic-embed-text	Creates local vector representations.
Storage	ChromaDB	Stores vectors, text, and metadata locally.
Retrieval	Semantic search	Finds relevant chunks and filters weak matches.
Reasoning	Ollama + Qwen3 8B	Generates answers from retrieved context.
API	FastAPI + Uvicorn	Backend interface for the future frontend.
Frontend	React/Vite	Planned user-facing interface.

5. What Is Not Fully Implemented Yet

- It is not yet a complete polished web product; frontend integration is still to be built.
- The FastAPI layer is at foundation stage; production API endpoints for chat, uploads, documents, status, and errors remain to be implemented.
- Persistent long-term conversation memory is planned but is not yet fully integrated into the current RAG flow.
- MCP-based actions are planned but are not yet integrated as a complete action system.
- PPTX multimodal understanding is postponed because slides can contain tables, images, charts, and other content beyond simple text.
- Chunking, relevance thresholds, source formatting, ingestion idempotency, and error handling still need polish.
- Authentication, permissions, deployment hardening, and production security are not yet part of the current prototype.

6. Future Roadmap

Stage	Milestone	Planned work
1	FastAPI backend	Define APIs for asking questions, uploading files, listing documents, health/status, and errors.
2	Frontend	Build React/Vite UI and connect it to FastAPI.
3	Streaming	Stream generated answers progressively into the UI.
4	Memory	Add persistent conversation/user memory across sessions.
5	Knowledge base	Improve document management, re-indexing, deletion, duplicate handling, and retrieval quality.
6	Actions / MCP	Connect controlled tools so VIBKED can perform useful actions.
7	Agentic reasoning	Add planning, tool selection, verification, and multi-step execution.
8	Multimodal knowledge	Later support richer PPTX content, OCR/vision, tables, images, and charts.
9	Private AI OS	Unify documents, memory, reasoning, and actions into one private workspace.
10	Polish & demo	Improve UX, reliability, source display, setup, observability, and performance.

7. Intended Future VIBKED Flow

DATA → KNOWLEDGE → MEMORY → REASONING → ACTION

Data enters locally → knowledge is extracted and indexed → useful memory persists → reasoning retrieves and interprets information → controlled tools/MCP allow actions. FastAPI and the frontend expose this as one user-facing system.

8. Self-Awareness Rule

This document can be ingested into VIBKED as a capability/roadmap document. When answering questions about itself, VIBKED should treat the current-capabilities section as implemented functionality and the roadmap as future intent. Planned features must not be presented as completed unless the project state has been updated.

Generated from the current VIBKED project state and development plan.